Algorithms and Complexity

A problem specifies a relationship between some input value(s) and some output value(s), i.e. which outputs are correct for every input instance.
An algorithm is a computational procedure that takes in the input value(s) and outputs some output value(s).
An algorithm is correct if, for every input, it halts in finite time, and produces the correct output as defined in the problem. A correct algorithm solves the problem.

Time Complexity

The time complexity of an algorithm is expressed in terms of the number of basic operations used by the algorithm for a particular size of input. The largest number of basic operations for a particular input size is the worst case time complexity.

Basic operations include additions, multiplications, comparisons, swaps and assignments.

- f is $O(g)$ if there are constants C and K such that:

  $|f(x)| \leq C|g(x)|$ for all $x \geq K$

Sum rule: if f_1 is $O(g_1)$ and f_2 is $O(g_2)$, then f_1+f_2 is $O\left(\{\max\{|g_1(x)|, |g_2(x)|\}\}\right)$

Product rule: if f_1 is $O(g_1)$ and f_2 is $O(g_2)$, then $f_1 \times f_2$ is $O(g_1(x) \cdot g_2(x))$

- f is $\Omega(g)$ if there are positive constants C and K such that:

  $|f(x)| \geq C|g(x)|$ for all $x > K$

  or equivalently, iff g is $O(f)$.

- f is $\Theta(g)$ if f is $O(g)$ and f is $\Omega(g)$.
- f is $o(g)$ if $\lim_{x\to\infty} \frac{f(x)}{g(x)} = 0$
- f is $\omega(g)$ if g is $o(f)$, i.e. $\lim_{x\to\infty} \frac{g(x)}{f(x)} = 0$

The running time $T(I)$ of an algorithm α on input I is the time α takes to halt on input I.
The worst-case running time $T(n)$ of α on inputs of size n is defined as:

$$T(n) = \max\{T(I) \mid I \text{ has size } n\}$$

i.e. for any input of size n, $T(I) \leq T(n)$

Advantages of asymptotic notation:

- difficult to be precise - A.N. gives clear upper bounds
- not dependent on a specific implementation
- we care about the inherent performance on large inputs.

Disadvantage

- we lose information - there can be large constants that get ignored.

Greedy Algorithms

In optimisation problems, we are trying to minimise or maximise some value. In these problems, every solution has a value, and we aim to find an optimal solution - one with the largest or smallest possible value.

An algorithm for an optimisation problem takes a series of steps. A greedy algorithm makes the locally optimal choice at each step. This does not always lead to a globally optimal solution.

Example: 0-1 knapsack problem

Miscreant robs a shop

- there are n items
- each item i is worth £v_i and weight w_i kg
- miscreant's knapsack can only hold W kg.

Which items to maximise profit?

Greedy strategy: work out the ratio $r_i = v_i / w_i$ and repeatedly take the remaining item with greatest r_i.

Running time: $\Theta(n \log n)$

Only produces an optimal solution in the fractional variant of the problem.

Divide and Conquer Algorithms

Many algorithms are recursive. At each level of recursion, they:

- Divide the problem into smaller subproblems
- Conquer the subproblems by solving them - could again be recursive.
- Combine the solutions to the subproblems into a solution for the original problem.

Recurrence relations are used to define the running time of these algorithms.

Example: Graph colouring problem.

The Chromatic Number χ_G of a graph G is the smallest integer for which G has a colouring where a colouring is an assignment of colours to the vertices of a graph such that no two adjacent vertices have the same colour.

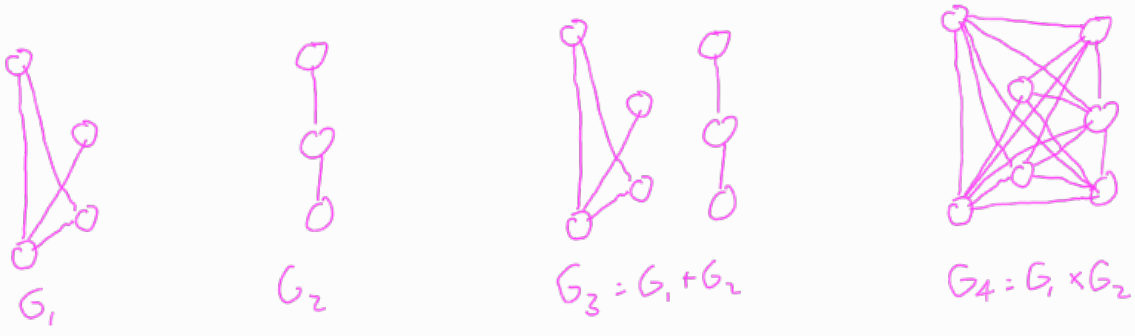
The problem of computing χ_G is NP-hard
The problem of determining whether $\chi_G \leq 3$ is also NP-hard.
However, if we restrict the input to Cographs, we can get a polynomial-time algorithm for both.

Note: determining whether $\chi_G \leq 2$ is trivial: Check all nodes, if there is a node with > 1 neighbour then false, else true.

Definitions:
For two graphs G_1 and G_2:

The disjoint union, $G_1 + G_2$ is the graph G_3 where:
- $V(G_3) = V(G_1) \cup V(G_2)$
- $E(G_3) = E(G_1) \cup E(G_2)$

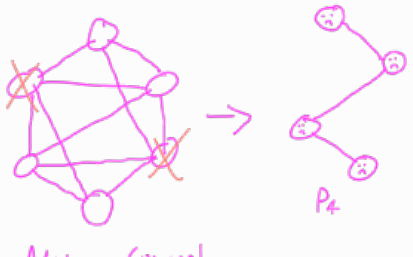
The join, $G_1 \times G_2$ is the graph G_4 where:
- $V(G_4) = V(G_1) \cup V(G_2)$
- $E(G_4) = E(G_1) \cup E(G_2) \cup E^*$
  where E^* is the set of all possible edges between a vertex in G_1 and a vertex in G_2



A Cograph is defined recursively as follows:
- the 1-vertex graph is a cograph.
- If G_1 and G_2 are cographs, so are $G_1 + G_2$ and $G_1 \times G_2$.

Any graphs that contain the subgraph P_4 = o-o-o-o are NOT cographs.
ALL graphs that are P_4-Free ARE Cographs.


Cographs



Not a Cograph

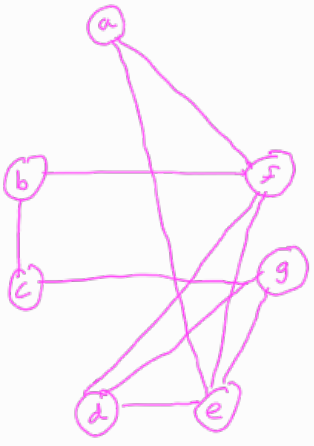
The Cotree T_G of a Cograph G is the unique tree that shows how G can be built up using joins and disjoint unions. A cotree has the following properties:

1. The root r of the tree represents the graph G
   - i.e. if we start at the bottom and work our way up the tree, once we reach r we have G.
2. Every leaf x of T_G represents a single vertex of G

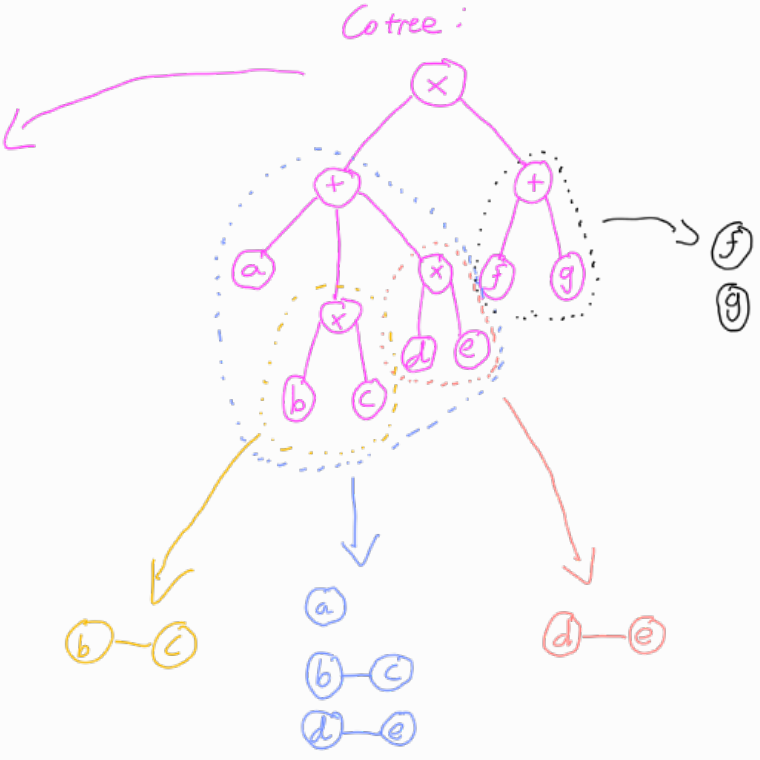
3. Every internal node x of T_G has at least two children, and is labelled either $+$ or $\times$, corresponding to the disjoint union or join of its children

4. Along the path from r to any leaf, the labels of the internal nodes must alternate between $+$ and $\times$. Note: this property ensures each cograph has a unique cotree.

Example Cograph:



Cotree:



Divide and Conquer Algorithm for Colouring Cographs

For a cograph G, construct its cotree T_G. This takes $O(|V|+|E|)$ time

Bottom-up approach - Start at the leaves, then process the parents etc.

Process nodes as follows:

- Leaves: leaves are single nodes of G, therefore their Chromatic Number χ is 1.
- Union-nodes: the chromatic number χ_{G_x} is the maximum of the chromatic numbers of the node's children, since they are disjoint graphs with no edges between them.
- join-nodes: the chromatic number χ_{G_x} is the sum of the chromatic numbers of all the node's children.

Total running time is polynomial
- $O(|V|+|E|)$ to build the tree
- $O(n)$ nodes in T_G
- $O(n)$ time per node.

Other examples of Divide and Conquer include:
- Merge Sort
- Maximum Subarray problem
- Strassen's algorithm for matrix multiplication.
  - uses 7 multiplications at each recursive step when $n \neq 1$, instead of 8 in the naive method.

Dynamic Programming

Dynamic Programming is an algorithmic technique used when naive recursive solutions to a problem are slowed down by repeatedly solving the same subproblems at each recursive step.
→ e.g. the naive solution to finding the first n fibonacci numbers will recalculate the i^{th} fibonacci number to work out the $(i+1)^{th}$ number etc.

Dynamic Programming arranges the subproblems in order to solve each one exactly once, storing the solution to each subproblem in memory – trading memory for time.

Note: for this to work, each subproblem must be independent, i.e. the solution to one subproblem doesn't affect another subproblem.

Dynamic Programming algorithms run in polynomial time if the number of distinct subproblems is polynomial, and each subproblem can be solved in polynomial time.

There are two approaches to Dynamic Programming:
- Top-down with memoisation: as basic recursion, but we store each new result in an array, and check if we already have the solution to each subproblem at each recursive step.
- Bottom-up: start with the 'smallest' subproblems and keep building up larger ones until we get to the final problem.

Dynamic programming can be applied to:
- Rod cutting problem:
  Given a rod of length n and a table of prices p_i, $1 \le i \le n$ for which a piece of rod of length i can be sold, determine the maximum possible revenue you can make from cutting up the rod and selling the pieces.
  Naive approach: $O(e^n)$
  Dynamic Programming: $O(n^2)$
- Matrix Chain multiplication problem
  Given a chain of multiplyable matrices $A_1 A_2 \dots A_n$, find the optimal way of parenthesising the chain multiplication to minimise the total number of operations.
  Brute force: $O(e^n)$
  Dynamic Programming: $O(n^3)$
- Longest Common Subsequence problem
  Given a sequence $X = x_1, x_2, \dots, x_m$, another sequence $Z = z_1, z_2, \dots, z_k$ is a subsequence of X if for some $i_1 < i_2 < \dots < i_k$ we have $z_1 = x_{i_1}$, $z_2 = x_{i_2}$, …, $z_k = x_{i_k}$. Note: does not have to be consecutive. A common subsequence of two strings is a subsequence of both.
  E.g. if X = BARRYISANIDIOT and Y = GERALDISCOOL, possible common subsequences include:

  RAIO
  AISO
  AO

  The problem is to find the longest common subsequence of two input strings.
  Brute force: $O(e^m)$
  Dynamic Programming: $O(m+n)$ m = length of input X, n = length of input Y

Graph Matchings

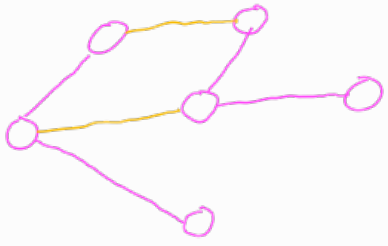
A matching M in an undirected graph $G=(V,E)$ is an independent set of edges where no two edges in M share a common vertex. Note: $M \subseteq E$

Trivial matchings include a single edge $e \subseteq E$, or the empty set of edges $\emptyset$.

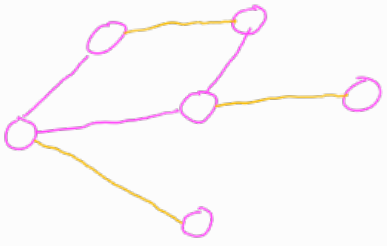
A matching M is:

- maximal if M is not a proper subset of any other matching in G, i.e. no other edges can be added to the matching.
- maximum if no other matching exists in G which contains a greater number of edges than M.
- perfect if every vertex of G is an endpoint of an edge in M.

A perfect matching is always maximum
A maximum matching is always maximal
A maximal matching is NOT always maximum.



Maximal
Not maximum
Not perfect.



Maximal
Maximum
perfect

Alternating Paths and Cycles

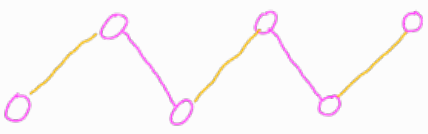
Consider a graph $G=(V,E)$ and a matching $M \subseteq E$.
Let $P=(v_0, v_1, \dots, v_k)$ be a path in G with edges $e_1 = v_0v_1, e_2 = v_1v_2, \dots, e_k = v_{k-1}v_k$.
P is M-alternating if $e_1, e_2, \dots, e_k$ are alternately in M and in $E \setminus M$

-> i.e. $e_1 \in M, e_2 \notin M, e_3 \in M, \dots$
or $e_1 \notin M, e_2 \in M, e_3 \notin M, \dots$

P is also M-augmenting if it is M-alternating and $e_1 \notin M$ and $e_k \notin M$, i.e. if M is NOT maximum over P.

Note: an M-augmenting path must consist of an odd number of edges.

Matching M



M-alternating

Matching M

M-augmenting

Similarly a cycle C in G is M-alternating if its edges alternately belong to M and $E \setminus M$.
Note: such a cycle must have an even number of edges.
M-augmenting cycles cannot exist.

It can be proven that a matching M for a graph G is maximum if and only if there exists no M-augmenting path.

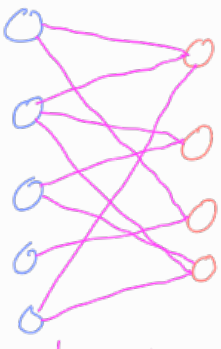
Furthermore, if G contains an M-augmenting path P, the matching $M \otimes P$ will contain exactly one more edge than M.

Note: $M \otimes P$ = the set of all edges either in M or P but not both (XOR).

To find a maximum matching for a graph, we can apply the following algorithm:

1. Let $M = \emptyset$
2. Check for an M-augmenting path P
3. If P exists, set $M = M \otimes P$ and repeat Step 2.
   Else, output M.

Note: we only consider bipartite graphs here, as Step 2 can be complicated otherwise.



a bipartite graph

bipartite graphs can be split into two groups of vertices such that no edge connects two vertices in the same group

Algorithm for finding augmenting paths in bipartite graphs:

Consider a bipartite graph G which can be partitioned into two (possibly empty) vertex sets R and B such that $|R| \leq |B| \Rightarrow |R| \leq n/2$

Let $M = \emptyset$

Check for an M-augmenting path P:

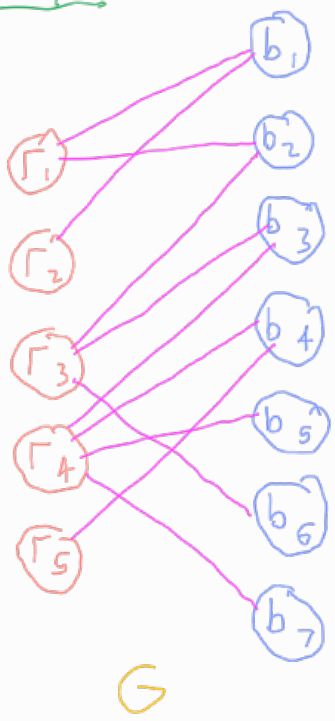
1. Start with an unmatched vertex $v \in R$
2. Consider all blue neighbours of v – if any of these are also unmatched, we have found P.
3. Otherwise, consider the unique matched red neighbour for each blue vertex found in 2.
4. For all red vertices found in 3., consider all new blue neighbours of these vertices. If any are unmatched, we have found P. If no new blue neighbours exist, terminate.
   Otherwise, back to Step 3.

If P has been found, set $M = M \otimes P$ and restart from Step 1.

If we terminated at Step 4 due to the lack of blue neighbours, then we found an alternating tree T with no augmented paths. We can safely consider $G - T$ and repeat the process until no P is found or there are no red vertices left.

If no P was found and we did not terminate due to a lack of neighbours, then M is a maximum matching for G.

Example



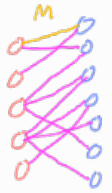
G

we have $\{r_1, \dots, r_5\} \in R$ and $\{b_1, \dots, b_7\} \in B$

Set $M = \emptyset$

1. Consider r_1
2. The blue neighbours of r_1 are b_1 and b_2. $b_1 \notin M$ so we have found an M-augmented path P

Set $M = M \otimes P = \{r_1 b_1\}$



1. Consider r_2
2. The only blue neighbour of r_2 is b_1
3. b_1 is matched so consider its matched neighbour r_1
4. Consider new blue neighbours of r_1 (excluding b_1). This is b_2. b_2 is unmatched so we have found an M-augmenting path P.

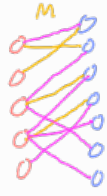
Set $M = M \otimes P = \{r_2 b_1, r_1 b_2\}$

1. Consider r_3.
2. The blue neighbours of r_3 are b_2, b_3, b_6. b_3 is unmatched so we have our P.

Set $M = \{r_2 b_1, r_1 b_2, r_3 b_3\}$

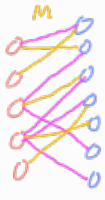
1. Consider r_4
2. The blue neighbours of r_4 are b_3, b_4, b_5, b_7 b_4 is unmatched — we found a P.

Set $M = \{r_2 b_1, r_1 b_2, r_3 b_3, r_4 b_4\}$



1. Consider r_5
2. the blue neighbour of r_5 is b_4
3. b_4 is matched so consider r_4
4. the new blue neighbours of r_4 are b_3, b_5, b_7 b_5 is unmatched — we have a P

Set $M = M \otimes P = \{r_2 b_1, r_1 b_2, r_3 b_3, r_5 b_4, r_4 b_5\}$



No more unmatched red vertices => we have our Maximum matching.

General version of the search algorithm for M-augmenting paths:

- for every vertex $v \in V$, search for an M-augmenting path starting at v (makes sense only if v is not matched in M)
- Definition: a vertex u is M-reachable from v if there is an M-alternating path from v to u

$\Rightarrow$ use a search algorithm to find all M-reachable vertices from v

- if we find an unmatched M-reachable vertex u then: M-augmenting path from v to u
- otherwise: no M-augmenting path from v

$\Rightarrow$ try other vertices v'

Search algorithm:

- grow a search tree rooted at v
- each path from v to any vertex u in the tree is alternating

$\Rightarrow$ alternating tree T

- vertices that we visit: labeled odd/even (otherwise unlabaled)
- vertex u is odd/even: the path from v in T is odd/even
- an unmatched vertex is labeled odd $\Rightarrow$ augmenting path
- the algorithm maintains a LIST of labeled nodes; initially LIST $= \{v\}$, label(v) = "even"
- iteratively remove a vertex from LIST and continue searching

Examining an even vertex u:

- scan its neighbors $N(u)$
- if $w \in N(u)$ is unlabeled: label(w) = "odd"; add w to LIST
- if w is unmatched $\Rightarrow$ augmenting path found; STOP

Examining an odd vertex u (which is matched):

- explore the unique edge $ux \in M$
- if x is unlabeled: label(x) = "even"; add x to LIST

The algorithm terminates if:

- either LIST is empty $\Rightarrow$ no augmenting path from v,
- or we assign "odd" label to an unmatched vertex $\Rightarrow$ augmenting path

This algorithm will always find an augmenting path (if it exists), if the graph has the unique label property.

A graph G has the unique label property with respect to a matching M and a root vertex v, if the search algorithm assigns a unique label (even/odd) to every labeled vertex, regardless of the order in which it visits the vertices.